

Retour d'expérience : TimescaleDB

Mickaël Boiziot

Contexte projet

- **SVOM : observatoire spatial franco-chinois pour détecter les sursauts gamma**
 - Constitué de plusieurs instruments dont la caméra ECLAIRs
 - Lancé depuis la Chine le 22 juin 2024
- **Eclairs Instrument Center**
 - En charge de l'instrument ECLAIRs
 - **Surveillance**, calibration, support scientifique et technique



Contexte projet

- **Surveillance de données de servitudes (House Keeping – HK)**
 - Bande, capteur, temps, valeur
- **3 bandes**
 - VHF : peu de données, toutes les 1.83s*, 8 paquets différents surveillés par l'EIC
 - Bande X : plus rare (~4-5h), données sur toute les secondes, plusieurs millions de HKs
 - Bande S
- **Plus de 800 HKs**
 - Environ 35M données uniques par jour
 - Entre 60 et 90 milliards de données uniques surveillées sur l'ensemble de la mission
 - Beaucoup de duplication

Technologies utilisées

- **Flask, Jinja, SQLAlchemy, Astropy, Celery**
- **Redis**
- **PostgreSQL**
- **Docker**

En pratique

- **4 tables principales**

- Même schéma : « id, band_id, hk_id, time, value »
- **hk_value_float** : value comme double precision
- **hk_value_integer** : value comme integer
- **hk_value_string** : value comme text
- **hk_value_ref** : value comme integer, plus une table secondaire pour faire la liaison

En pratique

- **Pas de stockage sous forme d'« un paquet = une table »**
 - Permet d'accéder simplement aux données issue de sources/paquets différents
 - Pas de cas spéciaux pour chaque HKs (IE : Pas de code Python spécifique pour chaque HK, interface unique)

La problématique

- Beaucoup plus de données
- « **id : 8 bytes**, **band_id : 2 bytes**, **hk_id : 2 bytes**, **time : 8 bytes**, **value :**
minimum 32 bytes »
 - Duplication de la donnée « time » et « band_id »
 - Column « id » utile pour le nettoyage des doublons mais inutile pour l'application
- **Taille des index importantes**
 - D'autant plus avec le nettoyage des doublons
- **1.7 To après 21 mois (données + index)**
- **Lenteur de certaines requêtes**

Contexte

- J'en avais parlé l'année dernière pendant l'ASOV/Semi
- **Guillaume Mella avait forwardé un e-mail sur un « Café SIST » au sujet de TimescaleDB**
 - Spoiler : Merci beaucoup !

Essais avant TimescaleDB

- **Partitionnement**
 - Gain vraiment très faible en terme de rapidité
- **Tweaking des paramètres de Postgres**
 - Gain important sur la rapidité mais insuffisant
 - Toujours le problème d'espace

TimescaleDB

- **Extension de Postgres dédiée aux données temporelles**
 - Partitionnement automatique
 - Possibilité de découpage secondaire (capteur!)
 - Compression automatique possible
 - Y compris après un délai variable
 - Plusieurs algorithmes, potentiellement combinés entre eux (delta, delta of delta, dict, count...)
- **Open-source**
- **Fournis des images docker**
- **Toujours du Postgres**
 - Compatible avec les drivers Postgres
 - Pas besoin de « tout ré-écrire »

Essais initiaux

- **Utilisation d'une partie des données**
 - Compression activée totalement
 - Vérification de l'avancement via l'espace sur le disque...
 - « Ça marche pas »
- **Gains extraordinaires**
 - ~800 Go => ~20 Go
 - Requête plusieurs minutes => < 1s
- **Oups...**
 - La compression c'est pas si simple
 - Saturation de l'espace disque
 - Données compressés ~= données figées

Essais initiaux et application

- **Plus de clé primaire auto increment « id »**
 - Pas possible avec les partitions
 - Nettoyage plus complexe des doublons
 - DELETE => SELECT DISTINCT / TABLE TEMPORAIRE / DELETE / INSERT
 - SQLAlchemy nécessite une clé primaire
 - Clé primaire sur plusieurs colonnes ?
 - Non fonctionnel (voir prochaine slide)
 - Upsert vs bulk insert
 - Pas d'utilisation d'ORM pour ces tables, utilisation de SQL directement
 - Fausse clé primaire

Clé primaire et anomalies

- **Remplacement de la clé primaire sur id vers une clé primaire multi colonne + upsert**

- band_id, hk_id, time : erreur
 - Découverte d'anomalies (EIC et instrument)
- Upsert (INSERT ou UPDATE si existant)
 - Pas « compatible » avec un bulk insert

> **Suppression de la clé primaire et nettoyage à posteriori**

Mise en production

- **Compression sur les données de plus de 3 semaines**
 - Permet de gérer les données en retard ou problèmes divers
- **Migration via « postgres_fdw »**
 - Double Postgres, connexion de la nouvelle instance sur l'ancienne
 - Importation des données « figées » dans un premier temps
 - Arrêt de la production puis importation des dernières données
 - Importation des données manquantes
 - Arrêt de courte durée

Résultat : gain d'espace

- **Gain d'espace conséquent**

- 1.7 To => 120 Go
- Les 3-4 semaines de données non compressées prennent autant d'espace que les 20 mois précédents compressés
 - Notamment à cause des index

- **Le nettoyage des données**

- DELETE => SELECT DISTINCT/TABLE TEMPORAIRE/DELETE/INSERT
 - Fait avoir un index avec beaucoup de données invalides et qui explose en taille
 - Réindexation plus fréquent mais aussi beaucoup plus rapide, sur les « chunks » / partitions
- Passage d'un cron système à un cron postgres via pg_cron

Résultat : gain d'espace

- Gain d'espace conséquent (1.7 To => 120 Go)

- Float :

bc_data	ac_data	pct_data	bc_index	ac_index	pct_index	bc_toast	ac_toast	bc_total	ac_total	pct_total
4572 MB	35 MB	0.76 %	1133 MB	3664 kB	0.32 %	0 bytes	331 MB	5704 MB	369 MB	6.48 %
4597 MB	35 MB	0.76 %	1140 MB	3688 kB	0.32 %	0 bytes	330 MB	5737 MB	369 MB	6.43 %
4258 MB	37 MB	0.87 %	1056 MB	3408 kB	0.32 %	0 bytes	277 MB	5314 MB	317 MB	5.97 %
4596 MB	35 MB	0.75 %	1139 MB	3688 kB	0.32 %	0 bytes	333 MB	5735 MB	372 MB	6.48 %
4525 MB	34 MB	0.75 %	1123 MB	3624 kB	0.32 %	0 bytes	329 MB	5648 MB	366 MB	6.49 %
4608 MB	35 MB	0.75 %	1141 MB	3696 kB	0.32 %	0 bytes	336 MB	5749 MB	374 MB	6.50 %

- Integer :

bc_data	ac_data	pct_data	bc_index	ac_index	pct_index	bc_toast	ac_toast	bc_total	ac_total	pct_total
1836 MB	40 MB	2.15 %	446 MB	1480 kB	0.32 %	0 bytes	3608 kB	2283 MB	45 MB	1.95 %
1928 MB	42 MB	2.18 %	468 MB	1560 kB	0.33 %	0 bytes	4552 kB	2396 MB	48 MB	2.00 %
1915 MB	42 MB	2.17 %	468 MB	1544 kB	0.32 %	0 bytes	1576 kB	2383 MB	45 MB	1.87 %
1411 MB	31 MB	2.16 %	342 MB	1136 kB	0.32 %	0 bytes	4424 kB	1753 MB	36 MB	2.05 %
1358 MB	30 MB	2.18 %	330 MB	1096 kB	0.32 %	0 bytes	3704 kB	1688 MB	34 MB	2.03 %
1951 MB	43 MB	2.18 %	473 MB	1568 kB	0.32 %	0 bytes	4664 kB	2424 MB	49 MB	2.01 %

Résultat : gain d'espace

- Gain d'espace conséquent

- Ref :

bc_data	ac_data	pct_data	bc_index	ac_index	pct_index	bc_toast	ac_toast	bc_total	ac_total	pct_total
1934 MB	50 MB	2.56 %	470 MB	1552 kB	0.32 %	0 bytes	6376 kB	2404 MB	57 MB	2.38 %
1932 MB	50 MB	2.56 %	469 MB	1552 kB	0.32 %	0 bytes	6280 kB	2402 MB	57 MB	2.38 %
1931 MB	49 MB	2.54 %	470 MB	1552 kB	0.32 %	0 bytes	6072 kB	2400 MB	56 MB	2.35 %
1930 MB	50 MB	2.57 %	471 MB	1552 kB	0.32 %	0 bytes	5664 kB	2401 MB	57 MB	2.36 %
1930 MB	50 MB	2.57 %	470 MB	1552 kB	0.32 %	0 bytes	5704 kB	2400 MB	57 MB	2.36 %
1930 MB	49 MB	2.51 %	470 MB	1552 kB	0.32 %	0 bytes	6504 kB	2400 MB	56 MB	2.35 %

- String :

bc_data	ac_data	pct_data	bc_index	ac_index	pct_index	bc_toast	ac_toast	bc_total	ac_total	pct_total
1836 MB	40 MB	2.15 %	446 MB	1480 kB	0.32 %	0 bytes	3608 kB	2283 MB	45 MB	1.95 %
1928 MB	42 MB	2.18 %	468 MB	1560 kB	0.33 %	0 bytes	4552 kB	2396 MB	48 MB	2.00 %
1915 MB	42 MB	2.17 %	468 MB	1544 kB	0.32 %	0 bytes	1576 kB	2383 MB	45 MB	1.87 %
1411 MB	31 MB	2.16 %	342 MB	1136 kB	0.32 %	0 bytes	4424 kB	1753 MB	36 MB	2.05 %
1358 MB	30 MB	2.18 %	330 MB	1096 kB	0.32 %	0 bytes	3704 kB	1688 MB	34 MB	2.03 %
1951 MB	43 MB	2.18 %	473 MB	1568 kB	0.32 %	0 bytes	4664 kB	2424 MB	49 MB	2.01 %

Résultat : rapidité

- **Assez variable**
 - Certaines pages qui n'ont pas eu de gain ou faible (usuellement celles avec beaucoup de requêtes séquentielles et/ou sur des données non compressés)
 - Des gains énormes sur certaines requêtes
 - Quelques dizaines de secondes/minutes => 1 seconde
 - Notamment sur les données compressés

Bonus « Nice to have »

- **Taille des backup divisée par 5 (~100Go => ~17Go)**
- **Possibilité d'utiliser ces backups pour le développement**
 - (1,7 To => 120 Go) donc possible sur un pc utilisateur
 - Permet d'avoir des données complètes pour le développement
 - Permet indirectement de tester le bon fonctionnement des backups